

Problemas k -ésimo Número Positivo Ausente e Contagem de Inversões

Prof. Thiago Braga Marcilon

Universidade Federal do Cariri

CC0058 – Tópicos Avançados em Algoritmos e Complexidade

Problema do k -ésimo Número Positivo Ausente

k -ésimo Número Positivo Ausente

Problema do k -ésimo Número Positivo Ausente

k -ésimo Número Positivo Ausente

- **Entrada:** um array A 0-indexado com n inteiros positivos distintos ordenados e um inteiro positivo k ;

Problema do k -ésimo Número Positivo Ausente

k -ésimo Número Positivo Ausente

- **Entrada:** um array A 0-indexado com n inteiros positivos distintos ordenados e um inteiro positivo k ;
- **Tarefa:** Determinar qual o k -ésimo inteiro positivo ausente de A ;

Ideias

Ideias

- Seja p_x o x -ésimo inteiro positivo ausente de A ;

Ideias

- Seja p_x o x -ésimo inteiro positivo ausente de A ;
- Pode acontecer $A[i] \leq i$?

Ideias

- Seja p_x o x -ésimo inteiro positivo ausente de A ;
- Pode acontecer $A[i] \leq i$? **Não!**

Ideias

- Seja p_x o x -ésimo inteiro positivo ausente de A ;
- Pode acontecer $A[i] \leq i$? Não!
- Então, o que podemos dizer se $A[i] = i + 1$?

Ideias

- Seja p_x o x -ésimo inteiro positivo ausente de A ;
- Pode acontecer $A[i] \leq i$? **Não!**
- Então, o que podemos dizer se $A[i] = i + 1$?
 - $A[i] < p_1 < \dots < p_k < \dots$;

Ideias

- Seja p_x o x -ésimo inteiro positivo ausente de A ;
- Pode acontecer $A[i] \leq i$? **Não!**
- Então, o que podemos dizer se $A[i] = i + 1$?
 - $A[i] < p_1 < \dots < p_k < \dots$;
- Estendendo o raciocínio anterior, o que podemos dizer quando comparamos i e $A[i]$?

Ideias

- Seja p_x o x -ésimo inteiro positivo ausente de A ;
- Pode acontecer $A[i] \leq i$? **Não!**
- Então, o que podemos dizer se $A[i] = i + 1$?
 - $A[i] < p_1 < \dots < p_k < \dots$;
- Estendendo o raciocínio anterior, o que podemos dizer quando comparamos i e $A[i]$?
 - Seja $d = A[i] - (i + 1)$;

Ideias

- Seja p_x o x -ésimo inteiro positivo ausente de A ;
- Pode acontecer $A[i] \leq i$? **Não!**
- Então, o que podemos dizer se $A[i] = i + 1$?
 - $A[i] < p_1 < \dots < p_k < \dots$;
- Estendendo o raciocínio anterior, o que podemos dizer quando comparamos i e $A[i]$?
 - Seja $d = A[i] - (i + 1)$;
 - $p_1 < \dots < p_d < A[i] < p_{d+1} < \dots$;

Ideias

Ideias

- Logo, se $d = A[i] - i - 1 \geq k$, $p_k < A[i]$, do contrário, $p_k > A[i]$;

Ideias

- Logo, se $d = A[i] - i - 1 \geq k$, $p_k < A[i]$, do contrário, $p_k > A[i]$;
- Podemos então aplicar uma *Busca Binária* em A

Ideias

- Logo, se $d = A[i] - i - 1 \geq k$, $p_k < A[i]$, do contrário, $p_k > A[i]$;
- Podemos então aplicar uma *Busca Binária* em A
- O objetivo é encontrar o índice x tal que $A[x] < p_k < A[x + 1]$ (se houver $A[x + 1]$);

Ideias

- Logo, se $d = A[i] - i - 1 \geq k$, $p_k < A[i]$, do contrário, $p_k > A[i]$;
- Podemos então aplicar uma *Busca Binária* em A
- O objetivo é encontrar o índice x tal que $A[x] < p_k < A[x + 1]$ (se houver $A[x + 1]$);
 - Se $k < A[0]$, $p_k = k$;

Ideias

- Logo, se $d = A[i] - i - 1 \geq k$, $p_k < A[i]$, do contrário, $p_k > A[i]$;
- Podemos então aplicar uma *Busca Binária* em A
- O objetivo é encontrar o índice x tal que $A[x] < p_k < A[x + 1]$ (se houver $A[x + 1]$);
 - Se $k < A[0]$, $p_k = k$;
 - Seja ini e fim os índices do subvetor em consideração, e $m = \lceil (ini + fim)/2 \rceil$;

Ideias

- Logo, se $d = A[i] - i - 1 \geq k$, $p_k < A[i]$, do contrário, $p_k > A[i]$;
- Podemos então aplicar uma *Busca Binária* em A
- O objetivo é encontrar o índice x tal que $A[x] < p_k < A[x + 1]$ (se houver $A[x + 1]$);
 - Se $k < A[0]$, $p_k = k$;
 - Seja ini e fim os índices do subvetor em consideração, e $m = \lceil (ini + fim)/2 \rceil$;
 - Se $d = A[m] - m \geq k$, procuraremos p_k no subvetor $A[ini \dots m - 1]$;

Ideias

- Logo, se $d = A[i] - i - 1 \geq k$, $p_k < A[i]$, do contrário, $p_k > A[i]$;
- Podemos então aplicar uma *Busca Binária* em A
- O objetivo é encontrar o índice x tal que $A[x] < p_k < A[x + 1]$ (se houver $A[x + 1]$);
 - Se $k < A[0]$, $p_k = k$;
 - Seja ini e fim os índices do subvetor em consideração, e $m = \lceil (ini + fim)/2 \rceil$;
 - Se $d = A[m] - m \geq k$, procuraremos p_k no subvetor $A[ini \dots m - 1]$;
 - Caso contrário, procuraremos por p_k no subvetor $A[m \dots fim]$;

Ideias

- Logo, se $d = A[i] - i - 1 \geq k$, $p_k < A[i]$, do contrário, $p_k > A[i]$;
- Podemos então aplicar uma *Busca Binária* em A
- O objetivo é encontrar o índice x tal que $A[x] < p_k < A[x + 1]$ (se houver $A[x + 1]$);
 - Se $k < A[0]$, $p_k = k$;
 - Seja ini e fim os índices do subvetor em consideração, e $m = \lceil (ini + fim)/2 \rceil$;
 - Se $d = A[m] - m \geq k$, procuraremos p_k no subvetor $A[ini \dots m - 1]$;
 - Caso contrário, procuraremos por p_k no subvetor $A[m \dots fim]$;
- Ao encontrar o índice x , o valor procurado é o $t = A[x] + (k - d)$ para $d = A[x] - (x + 1)$.

Ideias

- Logo, se $d = A[i] - i - 1 \geq k$, $p_k < A[i]$, do contrário, $p_k > A[i]$;
- Podemos então aplicar uma *Busca Binária* em A
- O objetivo é encontrar o índice x tal que $A[x] < p_k < A[x + 1]$ (se houver $A[x + 1]$);
 - Se $k < A[0]$, $p_k = k$;
 - Seja ini e fim os índices do subvetor em consideração, e $m = \lceil (ini + fim)/2 \rceil$;
 - Se $d = A[m] - m \geq k$, procuraremos p_k no subvetor $A[ini \dots m - 1]$;
 - Caso contrário, procuraremos por p_k no subvetor $A[m \dots fim]$;
- Ao encontrar o índice x , o valor procurado é o $t = A[x] + (k - d)$ para $d = A[x] - (x + 1)$, ou seja, $t = k + x + 1$.

Diferença na prática

Diferença na prática

- A complexidade do algoritmo com essa ideia é $O(\log n)$ e a complexidade do algoritmo linear é $O(n)$. Qual vocês usariam?

Diferença na prática

- A complexidade do algoritmo com essa ideia é $O(\log n)$ e a complexidade do algoritmo linear é $O(n)$. Qual vocês usariam?
- Na verdade, para valores de n até $\approx 10^7$, o algoritmo linear é competitivo;

Diferença na prática

- A complexidade do algoritmo com essa ideia é $O(\log n)$ e a complexidade do algoritmo linear é $O(n)$. Qual vocês usariam?
- Na verdade, para valores de n até $\approx 10^7$, o algoritmo linear é competitivo;
- Isso acontece por causa da memória cache que pode ser até dezenas de vezes mais rápido do que a memória RAM!

Diferença na prática

- A complexidade do algoritmo com essa ideia é $O(\log n)$ e a complexidade do algoritmo linear é $O(n)$. Qual vocês usariam?
- Na verdade, para valores de n até $\approx 10^7$, o algoritmo linear é competitivo;
- Isso acontece por causa da memória cache que pode ser até dezenas de vezes mais rápido do que a memória RAM!
- Para que a busca binária seja muito melhor:
 - ❶ o trabalho feito em cada iteração deve ser relativamente grande; e/ou
 - ❷ ou o tamanho da entrada deve ser na casa dos 10^8 ou até mesmo 10^9 ou superior;

Diferença na prática

- A complexidade do algoritmo com essa ideia é $O(\log n)$ e a complexidade do algoritmo linear é $O(n)$. Qual vocês usariam?
- Na verdade, para valores de n até $\approx 10^7$, o algoritmo linear é competitivo;
- Isso acontece por causa da memória cache que pode ser até dezenas de vezes mais rápido do que a memória RAM!
- Para que a busca binária seja muito melhor:
 - 1 o trabalho feito em cada iteração deve ser relativamente grande; e/ou
 - 2 ou o tamanho da entrada deve ser na casa dos 10^8 ou até mesmo 10^9 ou superior;
- Cuidado! Nesse 2º caso, a quantidade de dados de entrada é tão grande que o tempo de I/O pode dominar o tempo de computação;

Diferença na prática

- A complexidade do algoritmo com essa ideia é $O(\log n)$ e a complexidade do algoritmo linear é $O(n)$. Qual vocês usariam?
- Na verdade, para valores de n até $\approx 10^7$, o algoritmo linear é competitivo;
- Isso acontece por causa da memória cache que pode ser até dezenas de vezes mais rápido do que a memória RAM!
- Para que a busca binária seja muito melhor:
 - ❶ o trabalho feito em cada iteração deve ser relativamente grande; e/ou
 - ❷ ou o tamanho da entrada deve ser na casa dos 10^8 ou até mesmo 10^9 ou superior;
- Cuidado! Nesse 2º caso, a quantidade de dados de entrada é tão grande que o tempo de I/O pode dominar o tempo de computação;
- Quando isso ocorrer, prefira funções como `fgets` à `scanf` ou `cin`.

Problema de Contagem de Inversões

Contagem de Inversões

Problema de Contagem de Inversões

Contagem de Inversões

- **Entrada:** um array A com n inteiros;

Problema de Contagem de Inversões

Contagem de Inversões

- **Entrada:** um array A com n inteiros;
- **Tarefa:** Determinar o número de inversões de A , ou seja, quantos pares de posições (i, j) de A existem tais que $A[i] > A[j]$;

Ideias

Ideias

- O algoritmo de força bruta testa todos os pares e, portanto, é $O(n^2)$;

Ideias

- O algoritmo de força bruta testa todos os pares e, portanto, é $O(n^2)$;
- Visto que pode haver até $O(n^2)$ inversões, será que $O(n^2)$ é o melhor que podemos fazer?

Ideias

- O algoritmo de força bruta testa todos os pares e, portanto, é $O(n^2)$;
- Visto que pode haver até $O(n^2)$ inversões, será que $O(n^2)$ é o melhor que podemos fazer?
- E se usarmos divisão e conquista?

Ideias

- O algoritmo de força bruta testa todos os pares e, portanto, é $O(n^2)$;
- Visto que pode haver até $O(n^2)$ inversões, será que $O(n^2)$ é o melhor que podemos fazer?
- E se usarmos divisão e conquista?
 - $T(n) = 2T(n/2) + O(n^2)$

Ideias

- O algoritmo de força bruta testa todos os pares e, portanto, é $O(n^2)$;
- Visto que pode haver até $O(n^2)$ inversões, será que $O(n^2)$ é o melhor que podemos fazer?
- E se usarmos divisão e conquista?
 - $T(n) = 2T(n/2) + O(n^2) = O(n^2) :$ (

Ideias

- O algoritmo de força bruta testa todos os pares e, portanto, é $O(n^2)$;
- Visto que pode haver até $O(n^2)$ inversões, será que $O(n^2)$ é o melhor que podemos fazer?
- E se usarmos divisão e conquista?
 - $T(n) = 2T(n/2) + O(n^2) = O(n^2) :$ (
- Mas e se, enquanto aplica a divisão e conquista eu também for ordenando A ?

Ideias

Ideias

- Se temos duas metades ordenadas, podemos aplicar o Merge Sort enquanto resolvemos o problema;

Ideias

- Se temos duas metades ordenadas, podemos aplicar o Merge Sort enquanto resolvemos o problema;
- Contamos o número de inversões e ordenamos (em ordem decresc.) ambas as metades de forma recursiva;

Ideias

- Se temos duas metades ordenadas, podemos aplicar o Merge Sort enquanto resolvemos o problema;
- Contamos o número de inversões e ordenamos (em ordem decresc.) ambas as metades de forma recursiva;
- Tendo as duas metades ordenadas, como podemos contar o número de inversões (i, j) com i pertencendo à primeira metade e j à segunda?

Ideias

- Se temos duas metades ordenadas, podemos aplicar o Merge Sort enquanto resolvemos o problema;
- Contamos o número de inversões e ordenamos (em ordem decresc.) ambas as metades de forma recursiva;
- Tendo as duas metades ordenadas, como podemos contar o número de inversões (i, j) com i pertencendo à primeira metade e j à segunda?
 - Como as duas metades estão ordenadas, com i pertencendo à primeira metade e j à segunda, se $A[i] > A[j]$ então $A[i] > A[x]$ para todo $x \geq j$;

Ideias

- Se temos duas metades ordenadas, podemos aplicar o Merge Sort enquanto resolvemos o problema;
- Contamos o número de inversões e ordenamos (em ordem decresc.) ambas as metades de forma recursiva;
- Tendo as duas metades ordenadas, como podemos contar o número de inversões (i, j) com i pertencendo à primeira metade e j à segunda?
 - Como as duas metades estão ordenadas, com i pertencendo à primeira metade e j à segunda, se $A[i] > A[j]$ então $A[i] > A[x]$ para todo $x \geq j$;
- A complexidade do algoritmo com essa ideia é a mesma do Merge Sort, ou seja, $O(n \log n)$.